

1. Turing Machines

- **Definition:** A Turing Machine (TM) is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$.
 - **Design:** Infinite tape, read/write head, finite control.
 - **Example:** TM for $L = \{a^n b^n \mid n \geq 1\}$.
 - **Extensions:**
 - Multi-tape TM
 - Non-deterministic TM
 - Stay-put TM
 - **Church–Turing Thesis:** All algorithmically solvable problems can be solved by TM.
-

2. Recursive and Recursively Enumerable Languages

- **Recursive (Decidable):** TM halts for all inputs.
 - **Recursively Enumerable (RE):** TM halts for valid inputs; may loop on invalid ones.
 - **Closure Properties:**
 - Recursive: Union, Intersection, Complement
 - RE: Union, Intersection (not Complement)
-

3. Halting Problem

- **Definition:** Whether a TM halts on a given input.
 - **Result:** Undecidable using contradiction (diagonalization).
-

4. Rice's Theorem & Recursion Theorem

- **Rice's Theorem:** All non-trivial properties of TM languages are undecidable.
 - **Recursion Theorem:** TM can refer to its own description (used in self-replicating code/quines).
-

5. Complexity Classes

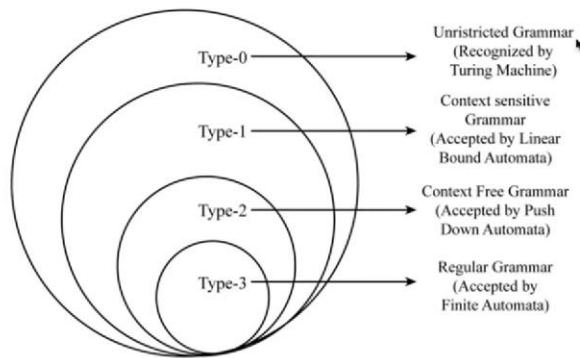
- **P:** Problems solvable in polynomial time.
 - **NP:** Problems verifiable in polynomial time.
 - **co-NP:** Complements of NP problems.
 - **PSPACE:** Polynomial space solvable problems.
 - **NPSPACE:** Equal to PSPACE (Savitch's Theorem).
-

6. Polynomial Time Reductions, NP-Completeness & NP-Hardness

- **Reduction ($\leq_p$):** Solving one problem using another in polynomial time.
- **NP-Complete:** In NP + every NP problem reduces to it.
 - Examples: SAT, 3-SAT, Clique, Vertex Cover.
- **NP-Hard:** As hard as NP-complete; not necessarily in NP.
 - Examples: Halting Problem, Tiling Problem.

Notes-YouTube

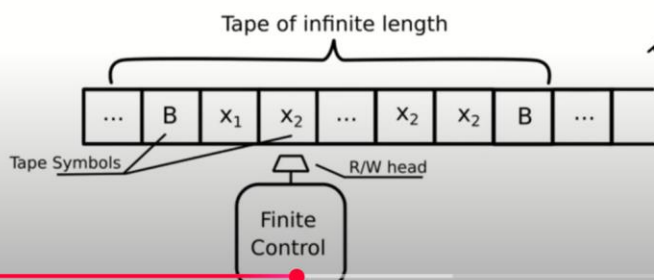
Introduction to Turing Machine



INTRODUCTION TO TURING MACHINE | TAFL | TOC | Automata Theory | UNIT 5 #tafl #toc #automatatheory

TMs are able to do any of the following Operations on tape

- 1) Read a symbol from the tape.
- 2) Write a symbol to the tape.
- 3) Move the tape head, one step LEFT.
- 4) Move the tape head, one step RIGHT.



Q: Is the Turing Machine tape infinite in both directions?

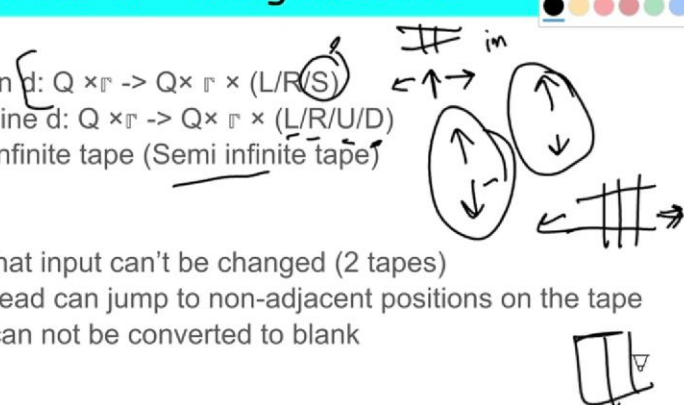
A: No, the standard Turing Machine tape is **infinite only in one direction** (usually to the right). However, there is a **variant** of the Turing Machine with a **doubly infinite tape** (infinite in both directions), but it is **not more powerful** than the standard one in terms of computational power.

Q: What does the Church–Turing Thesis state?

A: The Church–Turing Thesis states that any function that is computable by an algorithm can also be computed by a Turing Machine.

Modifications of Turing Machine

1. Multi-tape Turing Machine
2. Turing Machine with stay option d: $Q \times \Gamma \rightarrow Q \times \Gamma \times (L/R/S)$
3. Multi-dimensional Turing Machine d: $Q \times \Gamma \rightarrow Q \times \Gamma \times (L/R/U/D)$
4. Turing Machine with one way infinite tape (Semi infinite tape)
5. Multi Read/Write head points
6. Multi-head Turing Machine
7. Offline TM: have a restriction that input can't be changed (2 tapes)
8. Jumping TM: where the tape head can jump to non-adjacent positions on the tape
9. Non erasing TM: where input can not be converted to blank
10. Always writing TM
11. UTM
12. Non-Deterministic Turing Machine
13. TM without Writing Capacity (FA)
14. TM with Tape used as Stack
15. TM with finite tape
16. TM with Input Size Tape (LBA)



 **Q1: What is a Deterministic Turing Machine (DTM)?**

A: A DTM is a Turing Machine where **for each state and input symbol, there is exactly one transition**.

- **One path of execution** only.
- Works in a **predictable and fixed** manner.

 **Q2: What is a Non-Deterministic Turing Machine (NDTM)?**

A: An NDTM is a Turing Machine where **for some state and input symbol, there may be multiple possible transitions**.

- Can explore **many computation paths in parallel** (theoretically).
- Accepts input **if any path leads to acceptance**.

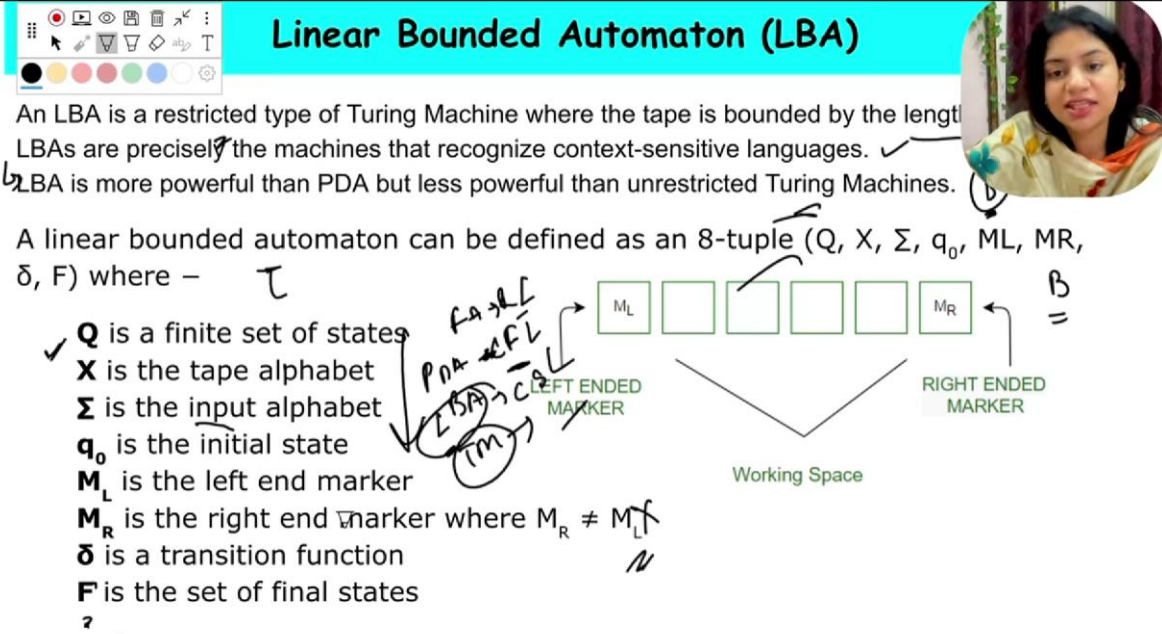
✦ **Note: NDTM and DTM have the same power** (they accept the same class of languages).

Q3: What is a Universal Turing Machine (UTM)?

A: A Universal Turing Machine is a Turing Machine that can **simulate any other Turing Machine**.

→ It takes as input the **description of another Turing Machine** and an **input string**, and simulates the behavior of that machine on that input.

→ **Key idea behind modern computers** — they can run any algorithm if programmed correctly.



Linear Bounded Automaton (LBA)

An LBA is a restricted type of Turing Machine where the tape is bounded by the length of the input string. LBAs are precisely the machines that recognize context-sensitive languages. LBA is more powerful than PDA but less powerful than unrestricted Turing Machines.

A linear bounded automaton can be defined as an 8-tuple $(Q, X, \Sigma, q_0, M_L, M_R, \delta, F)$ where —

- Q is a finite set of states
- X is the tape alphabet
- Σ is the input alphabet
- q_0 is the initial state
- M_L is the left end marker
- M_R is the right end marker where $M_R \neq M_L$
- δ is a transition function
- F is the set of final states

The diagram shows a tape with markers M_L and M_R and a 'Working Space' between them. Handwritten notes include 'PDA', 'LBA', 'TM', and 'FA' with arrows pointing to different parts of the diagram.

USICT GGSIPU ke syllabus ke hisaab se (jo tune diya hai), usme "Universal Turing Machine (UTM)" directly mentioned nahi hai.

✦ Kya Universal TM padhna zaruri hai?

Answer: Directly nahi, lekin indirectly useful ho sakta hai — kyunki:

1. **Halting Problem** aur **Rice's Theorem** me UTM ka concept background me use hota hai.
2. Agar **theoretical MCQ** ya **assertion-reason** type questions aaye, toh **UTM ka basic idea** helpful hoga.

🔍 Toh bas itna padhna kaafi hoga UTM ke liye:

✓ Universal Turing Machine (UTM) – Basic Idea

- **Definition:**

A Universal Turing Machine is a TM that can simulate **any other TM** on arbitrary input.

- **Motive:**

Just like a **general-purpose computer**, UTM can take as input:

- Description of a Turing Machine M
- Input w to that TM

And simulate the behavior of M on w .

- **Symbolically:**

$$U(\langle M, w \rangle) = M(w)$$

- **Why Important?**

- Proves that a **single TM** can be programmed to perform **any computation** — just like today's computers.
- Core idea behind **Halting Problem** proof.

 Exam Relevance:

Topic	UTM Usefulness
Turing Machines	Not directly
Halting Problem	🔥 Yes, used in proof
Rice's Theorem	🔥 Yes, indirectly
Recursion Theorem	Slightly
Complexity Classes	❌ Not used

 So Final Verdict:

UTM ka detailed design ya working yaad karne ki zarurat nahi hai, bas itna concept yaad rakho:

"Universal TM ek aisi machine hai jo kisi bhi Turing Machine ko simulate kar sakti hai."

Bilkul bhai! **Halting Problem** ka proof **important hai** — aur haan, **exam me aa sakta hai** in these forms:

- "Explain Halting Problem with proof."
- "Prove that the Halting Problem is undecidable."
- "Is Halting Problem recursively enumerable?"

So haan, **ye zarur padhna chahiye.**

Halting Problem – Full Proof (with Explanation)

 **Precise Proof (Same as Uploaded Solution):**

Halting Problem:

Given a Turing Machine M and input w , decide whether M halts on w .

Language:

$$\text{HALT} = \{(M, w) \mid M \text{ halts on } w\}$$

Proof:

Assume: There exists a Turing Machine H which decides HALT .

- Input to H : (M, w)
 - Output:
 - YES $\rightarrow$ if M halts on w
 - NO $\rightarrow$ if M does not halt on w
-

Construct a Machine H' using H :

- Input: (M, w)
- Working:
 - If H outputs YES $\rightarrow$ **Loop forever**
 - If H outputs NO $\rightarrow$ **Halt immediately**

(Thus, H' does the **opposite behaviour** of H .)

Now Construct Machine H' cap:

- Input: M
- Working:
 - Run H' on input (M, M)

(i.e., Machine MMM with its own description as input.)

Now run $H^\wedge$ on input $H^\wedge$ itself:

- Input: $H^\wedge$
- Working:
 - H' will be given $(H^\wedge, H^\wedge)$
 - Two cases arise:
 - If $H^\wedge$ halts on $H^\wedge \rightarrow H'$ will loop forever $\rightarrow$ contradiction.
 - If $H^\wedge$ does not halt on $H^\wedge \rightarrow H'$ halts $\rightarrow$ contradiction.

Thus, **contradiction occurs** in both cases.

Conclusion:

Hence, no such machine HHH can exist.

Therefore, the Halting Problem is Undecidable.

 **Final Mapping:**

Machine Meaning

H	Assumed decider for HALT
H'	Modified machine (opposite behaviour of H)
$H^\wedge$	Final machine to create contradiction

- ✓ Proof is exactly according to your uploaded solution.
- ✓ Short, exam-ready, same logic flow ($H \rightarrow H' \rightarrow \hat{H}$).
- ✓ Real standard explanation.

 **Short Answer (1-2 lines):**

The Halting Problem is undecidable.

This is proven by contradiction, assuming a decider H exists, constructing D that leads to paradox on input D(D).

